

✓ models.py

This file will handle loading the TinyLlama model and performing text generation.

```
import torch
from transformers import pipeline

class TinyLlamaModel:
    def __init__(self):
        self.pipeline = None
        self.load_model()

    def load_model(self):
        print("Loading TinyLlama model...")
        # You might need to specify the model path or name here
        # For example: "TinyLlama/TinyLlama-1.1B-Chat-v1.0"
        self.pipeline = pipeline(
            "text-generation",
            model="TinyLlama/TinyLlama-1.1B-Chat-v1.0",
            torch_dtype=torch.bfloat16,
            device_map="auto",
        )
        print("TinyLlama model loaded successfully.")

    def generate_text(self, prompt: str, max_new_tokens: int = 50) -> str:
        if not self.pipeline:
            self.load_model()

        sequences = self.pipeline(
            prompt,
            do_sample=True,
            top_k=50,
            top_p=0.95,
            num_return_sequences=1,
            max_new_tokens=max_new_tokens,
        )
        # The generated text includes the prompt, so we extract only the new part
        generated_text = sequences[0]['generated_text']
        return generated_text[len(prompt):].strip()

# Example usage (for testing within models.py if run directly)
if __name__ == "__main__":
    model_instance = TinyLlamaModel()
    test_prompt = "Write a short story about a brave knight and a dragon."
    response = model_instance.generate_text(test_prompt)
    print(f"Prompt: {test_prompt}")
    print(f"Response: {response}")
```

```

Loading TinyLlama model...
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:112: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/token)
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(

config.json: 100%                               608/608 [00:00<00:00, 18.2kB/s]
[transformers] `torch_dtype` is deprecated! Use `dtype` instead!
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits a
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_
model.safetensors: 100%                         2.20G/2.20G [00:50<00:00, 36.8MB/s]

Loading weights: 100%                           201/201 [00:00<00:00, 260.63it/s]

generation_config.json: 100%                    124/124 [00:00<00:00, 5.03kB/s]

tokenizer_config.json: 100%                     1.29k/1.29k [00:00<00:00, 98.5kB/s]

tokenizer.model: 100%                           500k/500k [00:00<00:00, 1.18MB/s]

tokenizer.json: 100%                           1.84M/1.84M [00:00<00:00, 28.2MB/s]

special_tokens_map.json: 100%                   551/551 [00:00<00:00, 45.0kB/s]
[transformers] Passing `generation_config` together with generation-related arguments=({'top_p', 'num_return_sequence
[transformers] Both `max_new_tokens` (=50) and `max_length` (=2048) seem to have been set. `max_new_tokens` will take
TinyLlama model loaded successfully.
[transformers] Ignoring clean_up_tokenization_spaces=True for BPE tokenizer LlamaTokenizer. The clean_up_tokenization
Prompt: Write a short story about a brave knight and a dragon.
Response:

```

▼ main.py

This file will set up the FastAPI application and define API endpoints for interacting with the TinyLlama model.

```

from fastapi import FastAPI
from pydantic import BaseModel
from typing import Optional

# Assuming models.py is in the same directory or accessible in the path
# For Colab, we can directly instantiate the class after running its cell
from __main__ import TinyLlamaModel

# Initialize the FastAPI app
app = FastAPI()

# Load the model globally to avoid reloading on each request
# This assumes the TinyLlamaModel class is already defined and executable in the Colab environment.
try:
    model_instance = TinyLlamaModel()
except Exception as e:
    print(f"Error initializing TinyLlamaModel: {e}")
    model_instance = None

# Define a request body schema for the /generate endpoint
class PromptRequest(BaseModel):
    prompt: str
    max_new_tokens: Optional[int] = 50

# Define the root endpoint
@app.get("/")
async def read_root():
    return {"message": "Welcome to the TinyLlama FastAPI server!"}

# Define the /generate endpoint
@app.post("/generate")
async def generate_response(request: PromptRequest):
    if model_instance is None:
        return {"error": "Model not loaded. Please check server logs.", "response": ""}
    try:
        response_text = model_instance.generate_text(request.prompt, request.max_new_tokens)
        return {"prompt": request.prompt, "response": response_text}
    except Exception as e:
        return {"error": str(e), "response": ""}

```

```
# To run this FastAPI app in Colab, you would typically use uvicorn.
# However, direct execution like 'uvicorn main:app --reload' is for local environments.
# In Colab, you can use a tool like 'nest_asyncio' and run it directly in a cell.
# For now, this defines the application structure.
```

```
Loading TinyLlama model...
```

```
Loading weights: 100%
```

```
201/201 [00:00<00:00, 2701.49it/s]
```

```
TinyLlama model loaded successfully.
```

To run the FastAPI server within Colab, you typically need to install `uvicorn` and `nest_asyncio`, and then run the application using `uvicorn.run()`.

First, install the necessary libraries:

```
!pip install fastapi uvicorn nest_asyncio
```

```
Requirement already satisfied: fastapi in /usr/local/lib/python3.12/dist-packages (0.136.3)
Requirement already satisfied: uvicorn in /usr/local/lib/python3.12/dist-packages (0.49.0)
Requirement already satisfied: nest_asyncio in /usr/local/lib/python3.12/dist-packages (1.6.0)
Requirement already satisfied: starlette>=0.46.0 in /usr/local/lib/python3.12/dist-packages (from fastapi) (0.52.1)
Requirement already satisfied: pydantic>=2.9.0 in /usr/local/lib/python3.12/dist-packages (from fastapi) (2.12.3)
Requirement already satisfied: typing-extensions>=4.8.0 in /usr/local/lib/python3.12/dist-packages (from fastapi) (4.
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from fastapi) (0.
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (from fastapi) (0.0.4)
Requirement already satisfied: click>=7.0 in /usr/local/lib/python3.12/dist-packages (from uvicorn) (8.4.1)
Requirement already satisfied: h11>=0.8 in /usr/local/lib/python3.12/dist-packages (from uvicorn) (0.16.0)
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from pydantic>=2.9.
Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib/python3.12/dist-packages (from pydantic>=2.9.0
Requirement already satisfied: anyio<5,>=3.6.2 in /usr/local/lib/python3.12/dist-packages (from starlette>=0.46.0->fa
Requirement already satisfied: idna>=2.8 in /usr/local/lib/python3.12/dist-packages (from anyio<5,>=3.6.2->starlette>
```

Now, you can run the FastAPI application. Note that running this cell will block execution until you stop the server. You'll typically run this in a separate thread or process when integrating with a Streamlit client in a local setup. For testing, you can run it and then interact with it using `requests` from another cell or a local client.

```
import uvicorn
import nest_asyncio

nest_asyncio.apply()

# This will run the FastAPI app on http://127.0.0.1:8000
# You can access it from outside Colab using ngrok or similar for public access, but for local testing, it's sufficient
# The 'app' object refers to the FastAPI instance defined in the previous cell.
# To run, uncomment the line below:
# uvicorn.run(app, host="0.0.0.0", port=8000)
```

▼ client.py

This file will implement the Streamlit chat interface to interact with the FastAPI backend.

```
!pip install streamlit -q
import streamlit as st
import requests

# Configuration for the FastAPI backend
FASTAPI_URL = "http://127.0.0.1:8000"
GENERATE_ENDPOINT = f"{FASTAPI_URL}/generate"

st.set_page_config(page_title="TinyLlama Chat")
st.title("🦜🔗 TinyLlama Chat Interface")

# Initialize chat history
if "messages" not in st.session_state:
    st.session_state.messages = []

# Display chat messages from history on app rerun
for message in st.session_state.messages:
    with st.chat_message(message["role"]):
        st.markdown(message["content"])
```

```
# React to user input
if prompt := st.chat_input("Say something to TinyLlama..."):
    # Display user message in chat message container
    st.chat_message("user").markdown(prompt)
    # Add user message to chat history
    st.session_state.messages.append({"role": "user", "content": prompt})

    with st.spinner("TinyLlama is thinking..."):
        try:
            # Send the prompt to the FastAPI backend
            response = requests.post(GENERATE_ENDPOINT, json={"prompt": prompt})
            response.raise_for_status() # Raise an HTTPError for bad responses (4xx or 5xx)
            api_response = response.json()

            # Extract the generated text
            generated_text = api_response.get("response", "Error: No response from model.")
            if api_response.get("error"): # Check for backend errors
                generated_text = f"Error from backend: {api_response['error']}"

        except requests.exceptions.ConnectionError:
            generated_text = "Error: Could not connect to the FastAPI server. Is it running?"
        except requests.exceptions.Timeout:
            generated_text = "Error: The request to the FastAPI server timed out."
        except requests.exceptions.RequestException as e:
            generated_text = f"An unexpected error occurred: {e}"

    # Display assistant response in chat message container
    with st.chat_message("assistant"):
        st.markdown(generated_text)
    # Add assistant response to chat history
    st.session_state.messages.append({"role": "assistant", "content": generated_text})

# Instructions for running Streamlit
st.sidebar.header("How to Run")
st.sidebar.markdown(
    "1. Ensure your FastAPI server is running in a separate process or terminal. "
    "In Colab, you would uncomment and run the `uvicorn.run(app, ...)` command in the `main.py` section."
)
st.sidebar.markdown(
    "2. In your local environment, save this code as `client.py` and run `streamlit run client.py` in your terminal."
    "In Colab, you would need to use `!streamlit run client.py` and expose the port using `ngrok` or `localtunnel`."
)

```

```

9.2/9.2 MB 52.4 MB/s eta 0:00:00
11.3/11.3 MB 76.6 MB/s eta 0:00:00
2026-06-19 21:58:07.671 WARNING streamlit.runtime.scriptrunner_utils.script_run_context: Thread 'MainThread': missing
2026-06-19 21:58:07.674 WARNING streamlit.runtime.scriptrunner_utils.script_run_context: Thread 'MainThread': missing
2026-06-19 21:58:07.982
Warning: to view this Streamlit app on a browser, run it with the following
command:

streamlit run /usr/local/lib/python3.12/dist-packages/colab_kernel_launcher.py [ARGUMENTS]
2026-06-19 21:58:07.984 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in ba
2026-06-19 21:58:07.985 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in ba
2026-06-19 21:58:07.985 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in ba
2026-06-19 21:58:07.987 Session state does not function when running a script without `streamlit run`
2026-06-19 21:58:07.988 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in ba
2026-06-19 21:58:07.989 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in ba
2026-06-19 21:58:07.990 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in ba
2026-06-19 21:58:07.991 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in ba
2026-06-19 21:58:07.993 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in ba
2026-06-19 21:58:07.993 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in ba
2026-06-19 21:58:07.995 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in ba
2026-06-19 21:58:07.996 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in ba
2026-06-19 21:58:07.996 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in ba
2026-06-19 21:58:07.997 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in ba
2026-06-19 21:58:07.999 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in ba
2026-06-19 21:58:08.000 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in ba
2026-06-19 21:58:08.001 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in ba
2026-06-19 21:58:08.002 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in ba
2026-06-19 21:58:08.003 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in ba
2026-06-19 21:58:08.004 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in ba
2026-06-19 21:58:08.005 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in ba
2026-06-19 21:58:08.005 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in ba
DeltaGenerator(_root_container=1, _parent=DeltaGenerator())

```

To run this Streamlit application effectively within Colab, you would typically need to expose the port where Streamlit runs (default 8501) to the internet using a service like `ngrok` or `localtunnel`.

First, install Streamlit and ngrok:

```
!pip install streamlit pyngrok
```

Next, you need to import `nest_asyncio` and `uvicorn` (if not already done) to manage the event loop, and then launch both the FastAPI app and the Streamlit app. For local testing, you would run the FastAPI server and the Streamlit client in separate terminals. In Colab, it's a bit more involved to run both concurrently and expose Streamlit.

Here's how you might approach it to run both the FastAPI and Streamlit apps, exposing Streamlit via ngrok. **Note: Running both in the same cell this way can be complex and might require careful management of processes.**

```
main.py is accessible here

server...)
0.0.0.0", port=8000)

it app...)
content to a file so Streamlit can run it
"w") as f:

1.0.1:8000"
API_URL}/generate"

le="TinyLlama Chat")
: Interface")

sion_state:
es = []

tate.messages:
ssage["role"]):
["content"])

"Say something to TinyLlama..."):
.markdown(prompt)
es.append({"role": "user", "content": prompt})

ama is thinking..."):

ests.post(GENERATE_ENDPOINT, json={"prompt": prompt})
for_status()
response.json()
= api_response.get("response", "Error: No response from model.")
.get("error"):
ext = f"Error from backend: {api_response['error']}"
ptions.ConnectionError:
= "Error: Could not connect to the FastAPI server. Is it running?"
ptions.Timeout:
= "Error: The request to the FastAPI server timed out."
ptions.RequestException as e:
= f"An unexpected error occurred: {e}"

ssistant"):
ed_text)
es.append({"role": "assistant", "content": generated_text})

up")
```

```

server is running in a separate process or terminal. "
comment and run the `uvicorn.run(app, ...)` command in the `main.py` section."

oment, save this code as `client.py` and run `streamlit run client.py` in your terminal."
eed to use `!streamlit run client.py` and expose the port using `ngrok` or `localtunnel`."

is a subprocess
ere because `streamlit run` itself is a blocking call.
amlit", "run", "client.py", "--server.port", "8501", "--server.enableCORS", "false", "--server.enableXsrfProtection",

te thread
Thread(target=run_fastapi)

'I a moment to start up

rate thread
g.Thread(target=run_streamlit)

to start and then expose it via ngrok
needed

ngrok authentication token
TOKEN" with your actual token from ngrok.com
NGROK_AUTH_TOKEN")

ie Streamlit port

ect(8501, "http")
c URL: {public_url}")
ove to access your Streamlit app.")
ers, interrupt this cell (Runtime -> Interrupt execution).")

start: {e}")
u have an ngrok auth token configured if required.")

e so the background threads continue to run
u might have a more robust way to manage this.
, this is often sufficient.
interrupt the cell to stop both servers.

```

```

from google.colab import userdata

# Retrieve the ngrok authentication token from Colab secrets
NGROK_AUTH_TOKEN = userdata.get('NGROK_AUTH_TOKEN')

# Check if the token was found
if NGROK_AUTH_TOKEN is None:
    print("Warning: NGROK_AUTH_TOKEN not found in Colab secrets. Please add it.")
else:
    ngrok.set_auth_token(NGROK_AUTH_TOKEN)
    print("ngrok authentication token set successfully.")

```

```

# Install pyngrok to ensure it's available, especially after kernel restarts.
!pip install pyngrok -q

import subprocess
import threading
import time
import nest_asyncio
import uvicorn
from pyngrok import ngrok
from google.colab import userdata

nest_asyncio.apply()

```

```

# Retrieve the ngrok authentication token from Colab secrets
NGROK_AUTH_TOKEN = userdata.get('NGROK_AUTH_TOKEN')

# Check if the token was found
if NGROK_AUTH_TOKEN is None:
    print("Warning: NGROK_AUTH_TOKEN not found in Colab secrets. Please add it to your Colab secrets manager under t
else:
    ngrok.set_auth_token(NGROK_AUTH_TOKEN)
    print("ngrok authentication token set successfully.")

def run_fastapi():
    # Make sure 'app' from main.py is accessible here
    global app
    print("Starting FastAPI server...")
    uvicorn.run(app, host="0.0.0.0", port=8000)

def run_streamlit():
    print("Starting Streamlit app...")
    # Write the client.py content to a file so Streamlit can run it
    with open("client.py", "w") as f:
        f.write("""
import streamlit as st
import requests

FASTAPI_URL = "http://127.0.0.1:8000"
GENERATE_ENDPOINT = f"{FASTAPI_URL}/generate"

st.set_page_config(page_title="TinyLlama Chat")
st.title("🦙 TinyLlama Chat Interface")

if "messages" not in st.session_state:
    st.session_state.messages = []

for message in st.session_state.messages:
    with st.chat_message(message["role"]):
        st.markdown(message["content"])

if prompt := st.chat_input("Say something to TinyLlama..."):
    st.chat_message("user").markdown(prompt)
    st.session_state.messages.append({"role": "user", "content": prompt})

    with st.spinner("TinyLlama is thinking..."):
        try:
            response = requests.post(GENERATE_ENDPOINT, json={"prompt": prompt})
            response.raise_for_status()
            api_response = response.json()
            generated_text = api_response.get("response", "Error: No response from model.")
            if api_response.get("error"):
                generated_text = f"Error from backend: {api_response['error']}"
        except requests.exceptions.ConnectionError:
            generated_text = "Error: Could not connect to the FastAPI server. Is it running?"
        except requests.exceptions.Timeout:
            generated_text = "Error: The request to the FastAPI server timed out."
        except requests.exceptions.RequestException as e:
            generated_text = f"An unexpected error occurred: {e}"

    with st.chat_message("assistant"):
        st.markdown(generated_text)
    st.session_state.messages.append({"role": "assistant", "content": generated_text})

st.sidebar.header("How to Run")
st.sidebar.markdown(
    "1. Ensure your FastAPI server is running in a separate process or terminal. "
    "In Colab, you would uncomment and run the `uvicorn.run(app, ...)` command in the `main.py` section."
)
st.sidebar.markdown(
    "2. In your local environment, save this code as `client.py` and run `streamlit run client.py` in your terminal. "
    "In Colab, you would need to use `!streamlit run client.py` and expose the port using `ngrok` or `localtunnel`."
)
""")

# This runs Streamlit as a subprocess
# We use a subprocess here because `streamlit run` itself is a blocking call.
subprocess.Popen(["streamlit", "run", "client.py", "--server.port", "8501", "--server.enableCORS", "false", "--s

```

```

# Start FastAPI in a separate thread
fastapi_thread = threading.Thread(target=run_fastapi)
fastapi_thread.start()

time.sleep(5) # Give FastAPI a moment to start up

# Start Streamlit in a separate thread
streamlit_thread = threading.Thread(target=run_streamlit)
streamlit_thread.start()

# Wait a bit for Streamlit to start and then expose it via ngrok
time.sleep(10) # Adjust as needed

# Open a ngrok tunnel to the Streamlit port
try:
    if NGROK_AUTH_TOKEN: # Only try to connect if token is available
        public_url = ngrok.connect(8501, "http")
        print(f"Streamlit Public URL: {public_url}")
        print("Click the URL above to access your Streamlit app.")
        print("To stop the servers, interrupt this cell (Runtime -> Interrupt execution).")
    else:
        print("ngrok tunnel not established because NGROK_AUTH_TOKEN was not found.")
except Exception as e:
    print(f"Ngrok failed to start: {e}")
    print("Please ensure you have an ngrok auth token configured if required.")

# Keep the main thread alive so the background threads continue to run
# In a real application, you might have a more robust way to manage this.
# For a Colab demonstration, this is often sufficient.
# You will need to manually interrupt the cell to stop both servers.
while True:
    time.sleep(1)

```

Project Details

Project Name: LLM FastAPI Serving Project

Repository Name: `llm-fastapi-streamlit-deployment`

Short Description: Production-ready AI deployment of a TinyLlama model using Python, FastAPI for RESTful APIs, and Streamlit for an interactive chat interface. Focuses on modularity, performance, and practical AI application (less than 300 characters).

LLM FastAPI Serving Project

Overview

This project demonstrates the transition from experimental notebook environments to production-ready AI deployment using Python, FastAPI, and Streamlit. It architects a modular system to serve a local TinyLlama model, creates RESTful API endpoints, builds an interactive chat interface, and evaluates real-world performance metrics. This project strengthens your ability to operationalize generative AI, design scalable backend-frontend integrations, and translate model capabilities into practical user-facing applications using the Discovery-to-Action (DTA) framework.

Project Goal

By the end of this project, the application should:

- Successfully run a modular FastAPI + Streamlit architecture serving a local LLM.
- Accept user prompts via a polished chat UI and return coherent, persona-aligned responses.
- Demonstrate clear understanding of backend-frontend communication, API design, and request/response handling.
- Identify performance bottlenecks and propose actionable optimization strategies for CPU-based inference.
- Articulate practical business or educational applications for the deployed chatbot.
- Provide a well-documented, reproducible repository ready for future cloud deployment or model swapping.

Architecture

The project is structured into three main modules to ensure a clear separation of concerns:

- `models.py`: Handles the loading and inference of the TinyLlama language model.

- `main.py`: Sets up a FastAPI application to expose the model's capabilities via a RESTful API.
- `client.py`: Implements a Streamlit-based interactive chat interface that communicates with the FastAPI backend.

```
graph TD
    User[User] --> Streamlit[Streamlit Client]
    Streamlit --> FastAPI[FastAPI Server]
    FastAPI --> TinyLlamaModel[TinyLlama Model - models.py]
    TinyLlamaModel --> FastAPI
    FastAPI --> Streamlit
    Streamlit --> User
```

Setup and Installation

Environment Configuration

1. Clone the Repository (if applicable):

```
git clone <your-repository-link>
cd llm-fastapi-streamlit-deployment
```

2. Install Dependencies: Ensure you have Python 3.8+ installed. Then, install the required packages:

```
pip install torch transformers fastapi uvicorn nest_asyncio streamlit requests pyngrok
```

(Note: For Colab, some installations are handled directly within the notebook cells.)

3. **Hugging Face Token (Optional but Recommended):** For better performance and to avoid rate limits when downloading models, consider setting your Hugging Face token. You can obtain one from [Hugging Face Settings](#).
4. **Ngrok Authentication Token:** To expose your Streamlit application publicly from Colab, you will need an `NGROK_AUTH_TOKEN`. Obtain one from [ngrok dashboard](#). Add this token to your Google Colab secrets manager under the name `NGROK_AUTH_TOKEN`.

Running the Application

1. `models.py` (Model Loading)

This file defines the `TinyLlamaModel` class, which loads the `TinyLlama/TinyLlama-1.1B-Chat-v1.0` model using `transformers` pipeline. It's designed to be instantiated once globally in the FastAPI application.

2. `main.py` (FastAPI Server)

This script sets up the FastAPI server with a `/generate` endpoint that accepts a user prompt and returns the model's response. The `TinyLlamaModel` is loaded once at startup to optimize performance.

API Design:

- **Endpoint:** `POST /generate`
- **Request Body:**

```
{
  "prompt": "Your input prompt here",
  "max_new_tokens": 50 (optional, default: 50)
}
```

- **Response Body (Success):**

```
{
  "prompt": "Your input prompt here",
  "response": "Generated text from TinyLlama"
}
```

- **Response Body (Error):**

```
{
  "error": "Error message",
}
```

```
"response": ""
}
```

To run the FastAPI server (in a separate terminal or Colab thread):

```
# This is typically run via uvicorn in a blocking manner.
# In a Colab environment, it's often started in a background thread.
import uvicorn
from main import app # Assuming 'app' is defined in main.py
uvicorn.run(app, host="0.0.0.0", port=8000)
```

3. `client.py` (Streamlit Chat Interface)

This script creates a conversational UI using Streamlit. It sends user prompts to the FastAPI backend and displays the responses. Error handling is included to manage connection issues with the backend.

To run the Streamlit client (in a separate terminal or Colab thread):

```
streamlit run client.py
```

Concurrent Execution in Colab

The provided Colab notebook demonstrates how to run both the FastAPI server and Streamlit client concurrently within the Colab environment using Python's `threading` module and `pyngrok` to expose the Streamlit interface.

1. **Ensure `NGROK_AUTH_TOKEN` is set** in Colab secrets.
2. **Run the dedicated Colab cell** that starts both FastAPI and Streamlit threads and initiates the ngrok tunnel. A public URL will be printed for accessing the Streamlit app.

Screenshot of Live Streamlit Interface:

 Streamlit Interface Screenshot (Please replace `<path-to-your-screenshot.png>` with an actual screenshot of your running Streamlit application.)

Bot Persona Customization

Custom System Prompt Configuration:

(This section is currently not implemented in the provided code. You would extend `models.py` or the `generate_text` function to accept a system prompt and integrate it into the model's input.)

To define your bot's persona (e.g., math tutor, code reviewer, domain specialist), you would typically include a `system_prompt` as part of the input to the LLM. This guides the model's behavior and response style. For example:

```
# Example modification in models.py (conceptual)
class TinyLlamaModel:
    # ... (existing code)
    def generate_text(self, prompt: str, system_prompt: str = "", max_new_tokens: int = 50) -> str:
        if system_prompt:
            full_prompt = f"""<|system|>
{system_prompt}<|end|>
<|user|>
{prompt}<|end|>
<|assistant|>
"""
        else:
            full_prompt = f"""<|user|>
{prompt}<|end|>
<|assistant|>
"""
        # ... (rest of the generation logic using full_prompt)
```